République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université M'hamed Bougara Boumerdes

Rapport de TP

Design Pattern : Adapter

Étudiants :
SLIMANI Faouzi
ZABOUR Wissal
Spécialité : Génie Logiciel — Master 1
Module : Architecture Logicielle
Encadrant : Dr. Imin
Date : 20 avril 2026

Année universitaire 2025 / 2026

Table des matières

1 Introduction générale **2**
1.1 Introduction 2
1.2 Problématique 2

2 Présentation des patrons de conception **4**
2.1 Définition 4
2.2 Types des patrons de conception 4

3 Pattern Adapter **6**
3.1 Définition 6
3.2 Problème 6
3.3 Solution 6
3.4 Composants d'un adaptateur 7
3.5 Avantages et Inconvénients 8
3.5.1 Avantages 8
3.5.2 Inconvénients 8

4 Implémentation **9**
4.1 Présentation du cas d'étude 9
4.2 Implémentation du pattern Adapter 9
4.2.1 Interface cible (Target) 9
4.2.2 Classe existante (Adaptee) 9
4.2.3 Classe Adapter 9
4.2.4 Classe Client 10
4.3 Explication 10

5 Conclusion **11**

Chapitre 1

Introduction générale

1.1 Introduction

Les design patterns, ou patrons de conception, sont des solutions réutilisables à des problèmes récurrents dans le développement logiciel. Ils permettent de structurer le code de manière efficace, en facilitant sa compréhension, sa maintenance ainsi que son évolution.

L'utilisation des design patterns présente plusieurs avantages majeurs. D'une part, ils favorisent la réutilisation du code en proposant des solutions éprouvées. D'autre part, ils améliorent la maintenabilité des applications en introduisant une organisation claire et modulaire. Enfin, ils facilitent la communication entre les développeurs grâce à un vocabulaire commun basé sur des concepts bien définis.

Parmi ces patrons, le pattern Adapter occupe une place importante dans la catégorie des patterns structurels. Il permet de faire collaborer des classes ayant des interfaces incompatibles, sans modifier leur code source. En d'autres termes, il agit comme un intermédiaire qui transforme l'interface d'une classe en une interface attendue par le client.

Ce pattern est particulièrement utile lors de l'intégration de composants externes, de bibliothèques tierces ou encore dans le cas de systèmes hérités (legacy systems), où il est nécessaire d'assurer la compatibilité entre différentes parties d'un logiciel.

1.2 Problématique

Dans le développement des systèmes logiciels, il est fréquent de devoir intégrer des composants existants ou des bibliothèques externes. Cependant, ces composants ne respectent pas toujours les mêmes interfaces que celles attendues par l'application principale, ce qui crée un problème d'incompatibilité.

En effet, une application peut être conçue pour fonctionner avec une interface spécifique, appelée interface cible (Target), tandis qu'un composant externe propose une interface différente, souvent incompatible (Adaptee). Cette divergence empêche une communication directe entre les deux parties, rendant leur intégration difficile, voire impossible sans modification du code existant.

Modifier le code source des composants existants n'est généralement pas une solution viable, notamment lorsque ces composants proviennent de bibliothèques tierces ou de systèmes hérités (legacy systems). Une telle modification peut introduire des erreurs, compromettre la stabilité du système ou rendre les mises à jour futures plus complexes.

Ainsi, la problématique principale est la suivante : comment permettre à deux classes ou systèmes ayant des interfaces incompatibles de collaborer efficacement, sans altérer

leur code source ?

Le pattern Adapter apporte une solution à ce problème en introduisant une classe intermédiaire qui assure la conversion de l'interface existante vers une interface attendue, permettant ainsi une interaction transparente entre les différentes composantes du système.

Chapitre 2

Présentation des patrons de conception

2.1 Définition

Les patrons de conception sont des solutions classiques à des problèmes récurrents de la conception de logiciels. Ce sont des sortes de plans ou de schémas que l'on peut personnaliser afin de résoudre un problème récurrent dans notre code.

2.2 Types des patrons de conception

Les patrons de conception (design patterns) se classent en trois grandes catégories selon leur but, regroupant généralement les 23 modèles classiques du « Gang of Four » (GoF) : création, structure et comportement. Ils améliorent la flexibilité, la réutilisation et la structure du code.

1. **Patrons de création (Creational)** : Facilitent l'instanciation des objets en encapsulant la logique de création.
2. **Patrons structurels (Structural)** : Assemblent des objets et classes en structures plus grandes tout en assurant leur flexibilité.
3. **Patrons comportementaux (Behavioral)** : Organisent les algorithmes, les communications et la répartition des responsabilités entre objets.

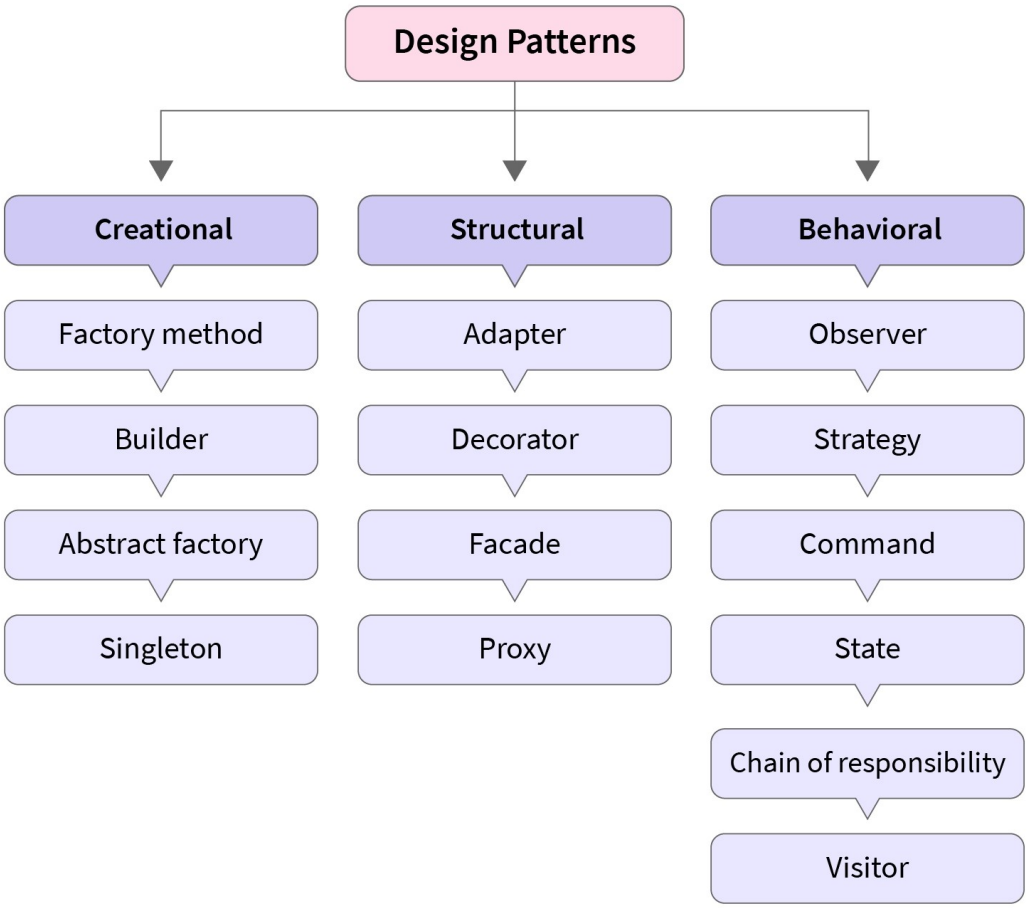

FIGURE 2.1 – Les différents types de design patterns

Dans le cadre de ce travail, nous nous intéressons particulièrement aux patrons structurels, et plus précisément au pattern Adapter. Ce dernier permet de résoudre les problèmes d'incompatibilité entre interfaces en introduisant une classe intermédiaire qui assure la communication entre différents composants logiciels.

Chapitre 3

Pattern Adapter

3.1 Définition

L'Adaptateur est un patron de conception structurel qui permet de faire collaborer des objets ayant des interfaces normalement incompatibles.

3.2 Problème

Deux composants logiciels doivent collaborer, mais leurs interfaces sont incompatibles. L'un attend des données dans un certain format (ex. JSON) alors que l'autre les fournit dans un format différent (ex. XML). Modifier l'un des composants est impossible ou risqué (code source inaccessible, risques de régression sur d'autres parties du système).

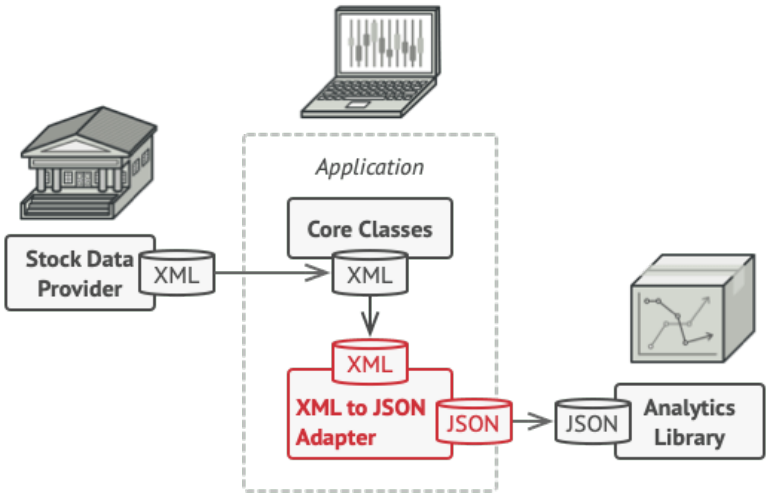

FIGURE 3.1 – Illustration du problème d'incompatibilité

3.3 Solution

L'adaptateur est un objet intermédiaire qui convertit l'interface d'un composant pour la rendre compatible avec celle attendue par l'autre. Il encapsule le composant existant, intercepte les appels, traduit les données ou les appels de méthodes dans le format et

l'ordre requis, puis transmet la requête au destinataire. Le composant encapsulé ignore l'existence de l'adaptateur, ce qui permet une intégration propre et sans effet de bord.

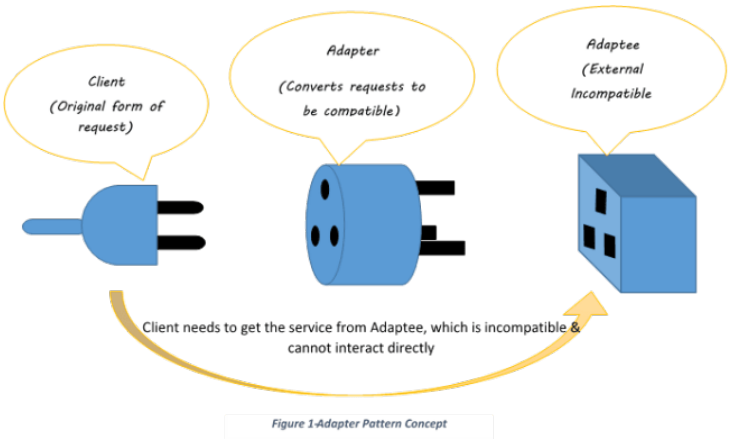

FIGURE 3.2 – Solution apportée par l'adaptateur

3.4 Composants d'un adaptateur

1. **Le client :** L'objet qui a besoin d'utiliser un service, mais qui attend une interface spécifique.
2. **La cible (Target) :** L'interface que le client comprend et attend. C'est le format ou le contrat que le client sait utiliser.
3. **L'adapté (Adaptee) :** L'objet existant que l'on veut utiliser, mais dont l'interface est incompatible avec le client. Il possède les fonctionnalités nécessaires, mais pas dans le bon format.
4. **L'adaptateur (Adapter) :** L'objet central qui implémente l'interface cible, traduit les appels du client en appels compréhensibles par l'adapté et convertit les données et l'ordre des opérations si nécessaire.

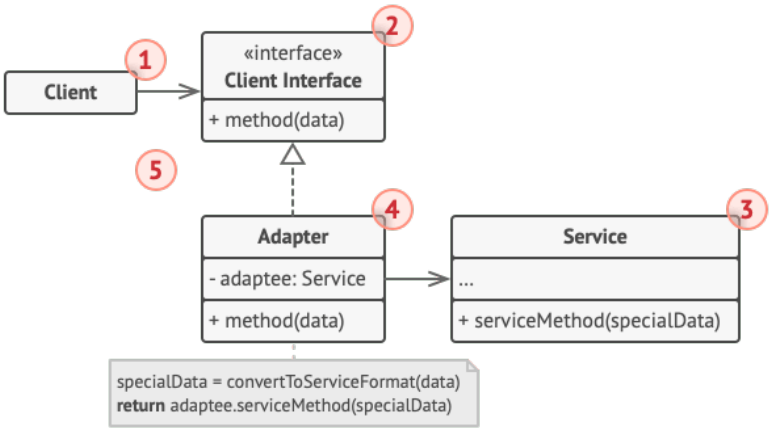

FIGURE 3.3 – Diagramme UML du pattern Adapter

3.5 Avantages et Inconvénients

3.5.1 Avantages

- Respect du principe d'ouverture/fermeture : on peut ajouter de nouveaux adaptateurs sans modifier le code existant du client ou de l'adapté.
- Séparation des préoccupations : la logique de conversion est isolée dans l'adaptateur, ce qui rend le code plus propre et maintenable.
- Réutilisabilité : permet d'utiliser des classes existantes (légataires, tierces, ou sans accès au code source) avec de nouvelles interfaces.

3.5.2 Inconvénients

- Complexité accrue : ajoute une couche supplémentaire au système, ce qui peut compliquer la compréhension et le débogage.
- Performance réduite : la conversion des données et la redirection des appels introduisent une légère surcharge (temps d'exécution et mémoire).

Chapitre 4

Implémentation

4.1 Présentation du cas d'étude

Dans cette partie, nous illustrons l'utilisation du pattern Adapter à travers un exemple de système de paiement. L'application est conçue pour utiliser une interface standard de paiement, tandis qu'un service externe (PayPal) possède une interface différente.

L'objectif est donc d'adapter ce service externe afin qu'il puisse être utilisé sans modifier le code existant.

4.2 Implémentation du pattern Adapter

4.2.1 Interface cible (Target)

```
interface Payment {
    void pay(int amountCents);
}
```

4.2.2 Classe existante (Adaptee)

```
class PaypalAPI {
    public void sendPayment(double amountInEuro) {
        System.out.println("Paiement via PayPal : " + amountInEuro + " €");
    }
}
```

4.2.3 Classe Adapter

```
class PaypalAdapter implements Payment {
    private PaypalAPI paypal;

    public PaypalAdapter(PaypalAPI paypal) {
        this.paypal = paypal;
    }

    @Override
    public void pay(int amountCents) {
```

```
        // Conversion des centimes en euros
        double amountInEuro = amountCents / 100.0;
        paypal.sendPayment(amountInEuro);
    }
}
```

4.2.4 Classe Client

```
public class Main {
    public static void main(String[] args) {
        Payment payment = new PaypalAdapter(new PaypalAPI());
        payment.pay(10000); // 10000 centimes = 100.00 €
    }
}
```

4.3 Explication

Dans cette implémentation, l'interface **Payment** représente l'interface attendue par le client. La classe **PaypalAPI** correspond au service externe dont l'interface est incompatible (elle attend un `double` en euros alors que le client fournit un `int` en centimes).

La classe **PaypalAdapter** joue le rôle d'intermédiaire en :

— Implémentant l'interface `Payment` attendue par le client.

— Encapsulant une instance de `PaypalAPI`.

— Convertissant les centimes en euros avant d'appeler `sendPayment()`.

Ainsi, le client peut utiliser le service PayPal sans connaître sa structure interne, ce qui illustre parfaitement le rôle du pattern Adapter. Il est également possible d'ajouter d'autres adaptateurs pour différents services de paiement (Stripe, Visa, etc.), ce qui montre la flexibilité du pattern.

Chapitre 5

Conclusion

Dans ce travail, nous avons étudié le pattern Adapter, un patron de conception structurel permettant de faire collaborer des classes aux interfaces incompatibles. Il constitue une solution efficace pour intégrer des composants existants sans modifier leur code.

Ainsi, le pattern Adapter améliore la flexibilité et la réutilisation des systèmes logiciels. Toutefois, son utilisation doit rester adaptée afin d'éviter une complexité inutile.

En somme, il s'agit d'un outil essentiel pour concevoir des applications modulaires et évolutives.